



CompSci 130

Introduction to Software Fundamentals

Simple Sorts (the $O(n^2)$ sorts) & Searching

Agenda & Reading

► Topics:

- Introduction
- Python built-in sorting functions
- “Simple” Sorting Algorithms
 - Selection Sort
 - Bubble Sort
 - Insertion Sort
- Searching (Linear & Binary Search)

► Further Study:

- Online Textbook: <https://onlinetextbook.auckland.ac.nz/04-sorting/>
- Demo: <http://math.hws.edu/eck/js/sorting/xSortLab.html>
 - selection, bubble, insertion (Click the Run button)
- Animation: <https://visualgo.net/en/sorting>





1 Introduction

Introduction

- ▶ **Sorting:** One of the Most Common Activities when using Computers!



Image source: Wikipedia

- ▶ **Examples:**
 - ▶ Alphabetically sorted names,
 - ▶ e.g. names in a telephone book, street names in a map index
 - ▶ Sorted numbers, e.g. house prices, student IDs, grades, rankings
 - ▶ Benefits:
 - Allows us to look for all students with A grades
 - Allows us to look for all houses within a certain price range etc.
 - Easy to find position of values in a sorted collection of items, e.g. minimum value, median value, quartile values



1 Introduction

Important Properties

- ▶ How efficient is the sorting algorithm?
 - ▶ Note: efficiency can depend on the order of the input data set, e.g. is it already almost sorted or completely unsorted (random)?
- ▶ How much additional memory does the sorting algorithm require?
- ▶ How easy is the sorting algorithm to implement?
 - ▶ For small data sets, simple sorting algorithms are often sufficient



1 Introduction

A comparison operator

- ▶ Any data which needs to be transformed to or kept in a sorted order will involve the **comparison** of items
- ▶ Example items:

- ▶ Numbers

`-34 < -1 < 0 < 1 < 245`

- ▶ Strings

`'A' < 'B' < 'C' ... < 'X' < 'Y' < 'Z'`

`'A' < 'B' < 'C' ... < 'a' < 'b' < 'c' ... < 'y' < 'z'`

`'Hungry' < 'Money' < 'More' < 'money' < 'work'`

- ▶ Note: When comparing keys which are strings, the ASCII (or Unicode) values of the characters of the string are used (e.g., 'a' is 97 and 'A' is 65).
- ▶ 'Money' < 'More' since 'n' < 'r'!



2 Python built-in

Python sorting – sorted()

- ▶ Python has a built-in sorting function: **sorted()**
 - ▶ This takes any sequence or object that is **iterable**, and **returns** a **new list** of the elements in sorted order
 - ▶ For example:

```
unsorted_numbers = [3, 8, 1, 2, 7, 6, 0, 5, 9, 4]
sorted_numbers = sorted(unsorted_numbers)
print(unsorted_numbers)
print(sorted_numbers)
print(unsorted_numbers == sorted_numbers)
```

Parameter:
A list of integers.

```
[3, 8, 1, 2, 7, 6, 0, 5, 9, 4]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
False
```



2 Python built-in

Python sorting – sorted()

- ▶ Python has a built-in sorting function: **sorted()**
 - ▶ This takes any sequence or object that is **iterable**, and **returns** a **new list** of the elements in sorted order
 - ▶ For example:

```
message = 'hello world'  
print(message)  
print(sorted(message))
```

Parameter: a string

hello world

[' ', 'd', 'e', 'h', 'l', 'l', 'l', 'o', 'o', 'r', 'w']

ASCII codes: 32 100 101 104 108 108 ...

A list of characters



2 Python built-in

Python Sorting – sort()

- ▶ An alternative to the built-in Python sorted() function is available solely for lists:
 - ▶ The sort() method can be used to sort the list elements **in place**.
 - ▶ That means, the order of the items in the original list will change!
 - ▶ For example:

```
numbers = [3, 8, 1, 2, 7, 6, 0, 5, 9, 4]
numbers.sort()
print('numbers = ', numbers)
```

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```




3 Sorting

Why sorting?

- ▶ Why should we study sorting, given that Python already provides functions for sorting?
 - ▶ So we have a detailed understanding of how different sorting algorithms work
 - ▶ Depending on the application, the use of different algorithms may be required to optimize for efficiency. For example, **for almost sorted collections of items**, the simple **insertion sort is very efficient**.
 - ▶ From an algorithm analysis point of view, we are interested in how much processing it takes to sort a collection of items (i.e., the Big-O).
 - ▶ The Python `sorted()` function and the `sort()` method use an $O(n \log n)$ algorithm called TimSort, which is a hybrid combining two well known algorithms: insertion sort and merge sort (will be examined later in this lecture)



3 Sorting

Sorting: Definitions & Assumptions

- ▶ Sorting is a process that organizes a collection of data into either *ascending* or *descending* order.
- ▶ Types of Simple ($O(n^2)$) Sorting Algorithms
 - ▶ Selection Sort
 - ▶ Bubble Sort
 - ▶ Insertion Sort
- ▶ Assumptions
 - ▶ The elements to be sorted
 - ▶ are placed in “array-like” sequences of length N
 - *array-like* means that indexing of elements is $O(1)$!
 - in the following examples, the array-like sequence we will consider is the Python list
 - ▶ might be integers, character strings, or other objects
 - ▶ contain a sort key (can be compared, e.g. ‘<’ for numbers)



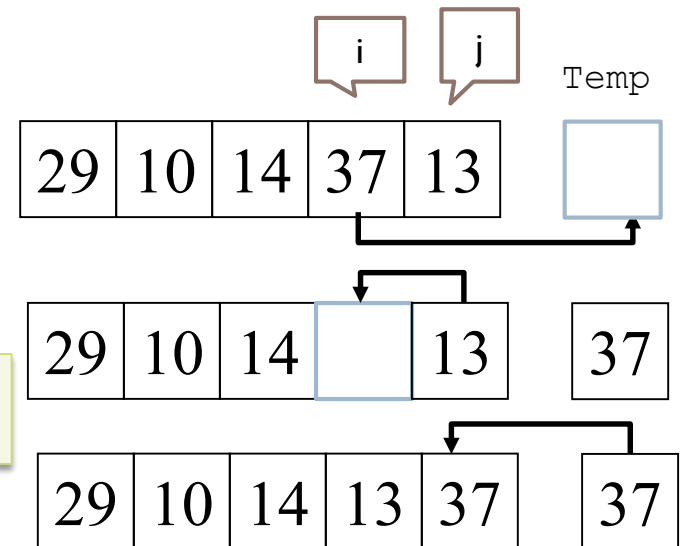
3 Sorting

The cost of sorting

- ▶ There are two operations that we usually consider when measuring the cost of sorting:
 - ▶ comparisons (comparing two items in a collection)
 - ▶ swaps or shifts (swapping or shifting items that are out of order)
 - ▶ How can we swap the values at position **i** and **j** in the list “**values**”?

```
def swap(values, i, j):  
    temp = values[i]  
    values[i] = values[j]  
    values[j] = temp
```

```
def swap2_compact(values, i, j):  
    values[i], values[j] = values[j], values[i]
```



- ▶ these operations affect the efficiency of a sorting algorithm (Big-O)!



3 Sorting

The cost of sorting

▶ Sorting “in place”

- ▶ We will consider sorting algorithms that operate “in place”, that is, except for a single temporary variable used when shifting items, we don’t require additional memory other than the memory for the sequence itself that is being sorted (memory complexity $O(1)$)

▶ A “pass”

- ▶ A pass is defined as one traversal through the list (or part of the list) comparing, and if necessary, swapping or shifting items along the way

▶ Ascending order

[3, 8, 1, 2, 7, 6, 0, 5, 9, 4]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

- ▶ We will sort items from smallest (on the left) to largest (on the right) element, but obviously sorting in the opposite direction is a trivial change



3 Sorting Complexity

- ▶ An algorithm can require different times to solve different problems of the same size.
 - ▶ Worst-case analysis
 - ▶ Find the maximum number of operations an algorithm requires given all possible situations
 - ▶ It is most commonly studied and easiest to calculate
 - ▶ Best-case analysis
 - ▶ Describe the way an algorithm behaves under optimal conditions (e.g. if the sequence is already sorted).
 - ▶ Average-case analysis
 - ▶ Expected runtime given arbitrary data
 - ▶ Hard to calculate but yields a more realistic behaviour in practice



4 Selection Sort

Selection Sort

- ▶ It works by selecting the largest element of the list and placing it at the end of the list. Then the process is repeated for the unprocessed portion of the list until the whole list is sorted.
 - ▶ Strategy
 - ▶ Selection sort divides the list into two parts: the **sorted** part, and the **unsorted part**.
 - ▶ In each pass of the list, selection sort finds the largest of the unsorted elements, and **swaps** it into its correct place. The size of the unsorted part is reduced by one and the size of the sorted part is increased by one.
 - ▶ The process is repeated until the unsorted part has a size of 1 – then all elements are sorted.
-



4 Selection Sort

Selection Sort

- ▶ Sorting numbers from left to right, smallest to largest

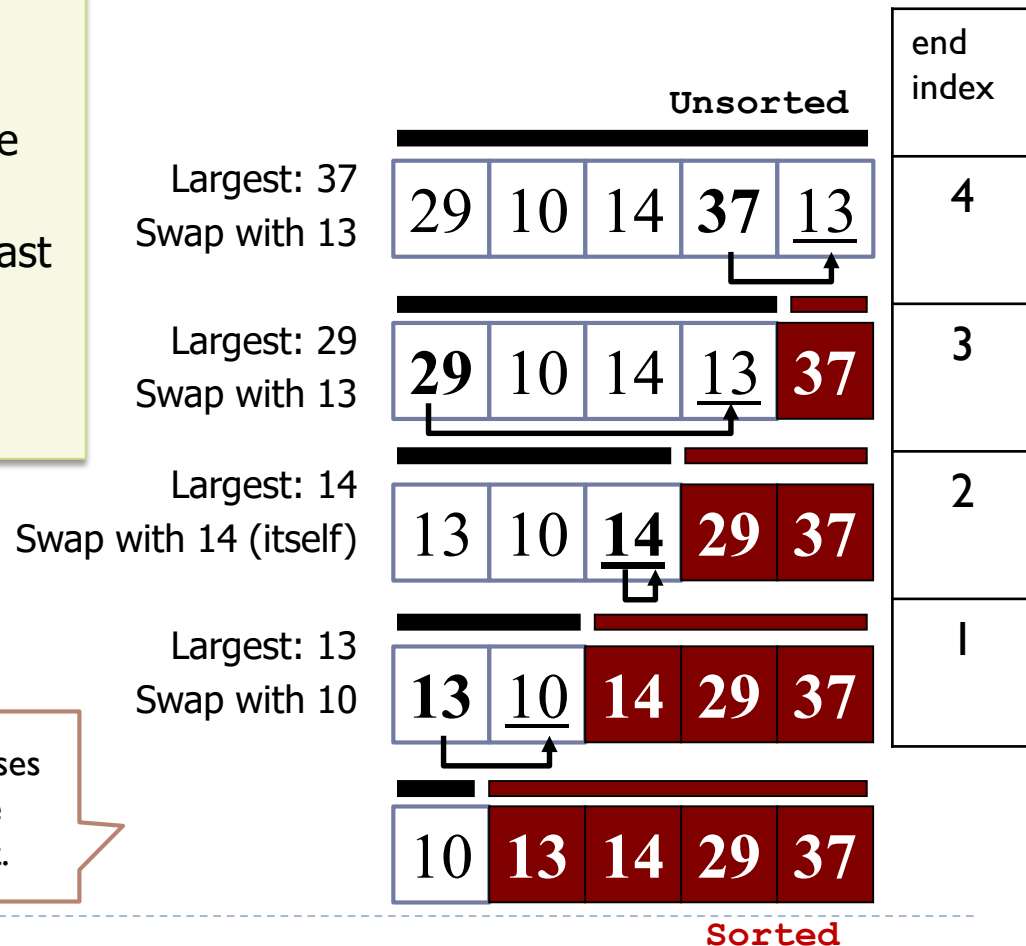
Algorithm:

while not finished

Search for the largest item in the unsorted part

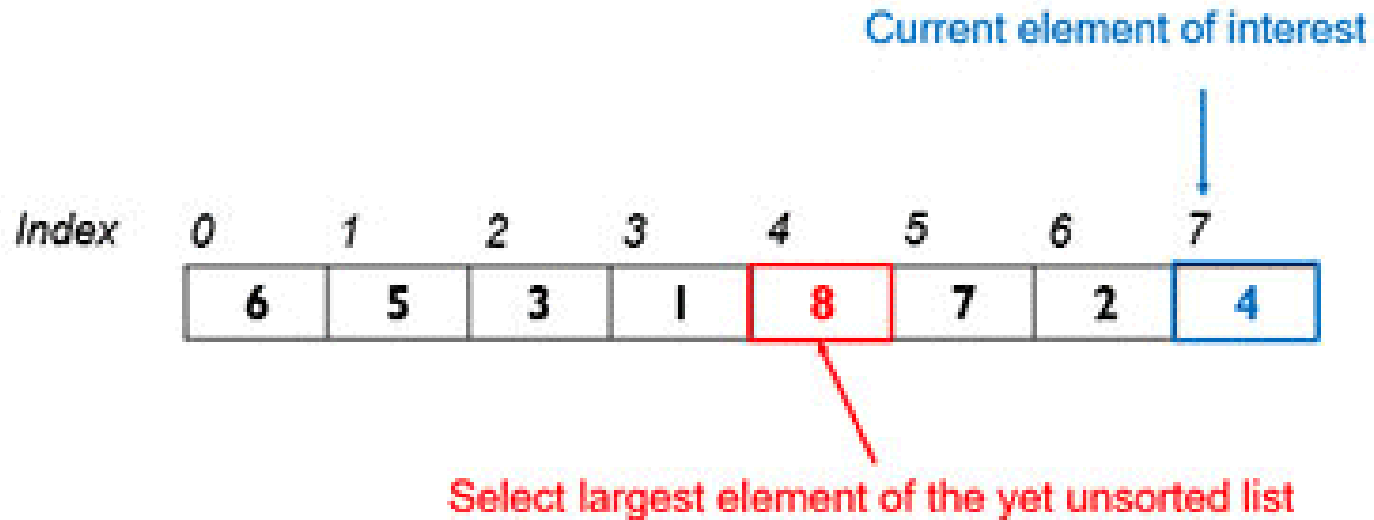
Swap the largest item with the last position of the unsorted list

Reduce the unsorted list by one element



The total number of passes required is 4 when there are 5 elements in the list.

Animation





4 Selection Sort

Exercise 1



► Sort the following numbers into ascending order.

► 12, 7, 24, 44, 19 and 32

Initial list	12	7	24	44	19	32
Largest: Swap with						
Largest: Swap with						
Largest: Swap with						
Largest: Swap with						
Largest: Swap with						

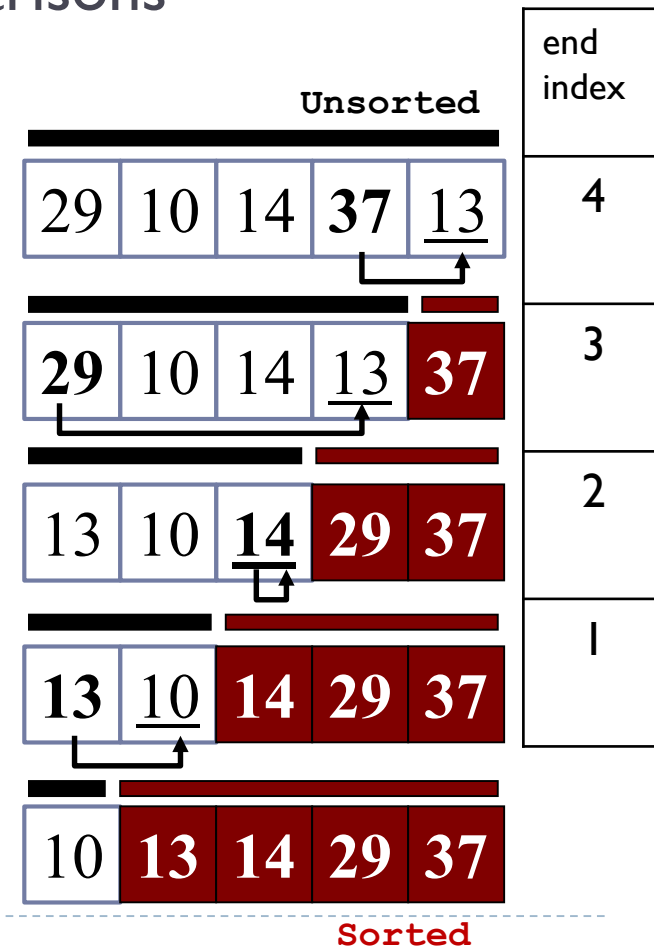


4 Selection Sort

Selection Sort

- ▶ What is the Big-O complexity?
 - ▶ On each pass, there are **several** comparisons and at most **one** swap.
 - ▶ The number of comparisons is data independent (initial order of elements is not relevant)

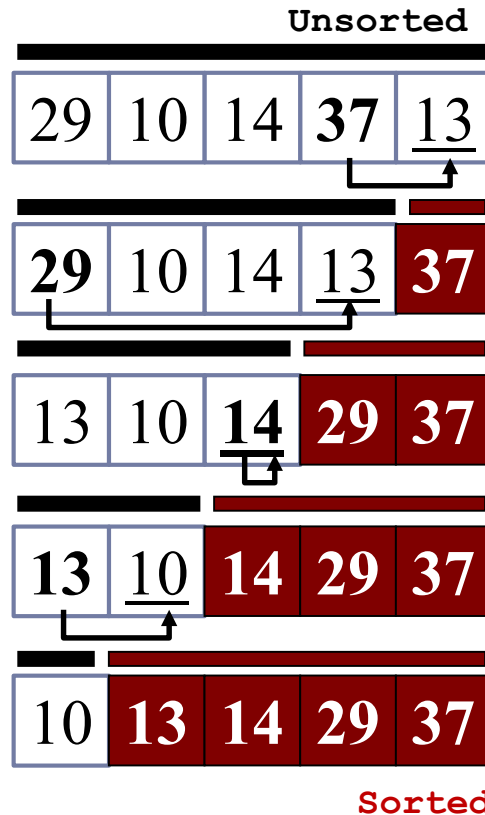
pass	#comp.	#swaps
1 st	4	1
2 nd	3	1
3 rd	2	1 (itself)
4 th	1	1





How to implement?

- ▶ For example: the number of elements = 5
- ▶ => 4 passes



number of
comparisons = 4

position_max	index	
0	1	
0	2	
0	3	update pos_largest
3	4	swap 37 and 13

number of
comparisons = 3

position_max	index	
0	1	
0	2	
0	3	swap 29 and 13

position_max	index	
0	1	
0	2	update pos_largest swap 14 and 14

number of
comparisons = 2

position_max	index	
0	1	swap 13 and 10

number of
comparisons = 1



4 Selection Sort Analysis

- ▶ What is the Big-O complexity?
 - ▶ $O(n^2)$
 - ▶ Best, average and worst case time complexities of Selection Sort are all $O(n^2)$, assuming that comparisons can be done in constant time.
- ▶ Advantage:
 - ▶ Simple and easy to implement
- ▶ Disadvantage:
 - ▶ Inefficient for large sequences



Exercise 2-3

- ▶ `print_elements_backward(numbers)`

- ▶ prints elements from the end to the first element

```
a_list = [29, 10, 14, 37, 13]
print_elements_backward(a_list)
```

13 37 14 10 29

- ▶ `get_index_of_largest(numbers, index)`

- ▶ returns the index of the largest element from the **first** element in the list to the element at the **specified index**.

```
a_list = [29, 10, 14, 37, 13]
print(get_index_of_largest(a_list, 2))
```

looks for the largest
element from [29, 10, 14]
= 0

```
a_list = [3, 15, 25, 37, 45]
print(get_index_of_largest(a_list, 3))
```

looks for the largest
element from [3, 15, 25, 37]
= 3



5 Bubble Sort

Bubble Sort

- ▶ It works by repeatedly stepping through the list to be sorted, **comparing two items** at a time, and **swapping these two items** if they are in the wrong order.
- ▶ Strategy
 - ▶ Bubble sort divides the list into two parts: the **sorted** part, and the **unsorted** part.
 - ▶ In each pass, the algorithm **compares adjacent elements** and **swaps** elements if they are not in the correct order => largest element is “bubbled” to the right of the unsorted part.
 - ▶ In each pass, the size of unsorted part is reduced by 1 and the size of the sorted part is increased by 1.
 - ▶ The process is repeated until the unsorted part has a size of 1 – then all elements are sorted.



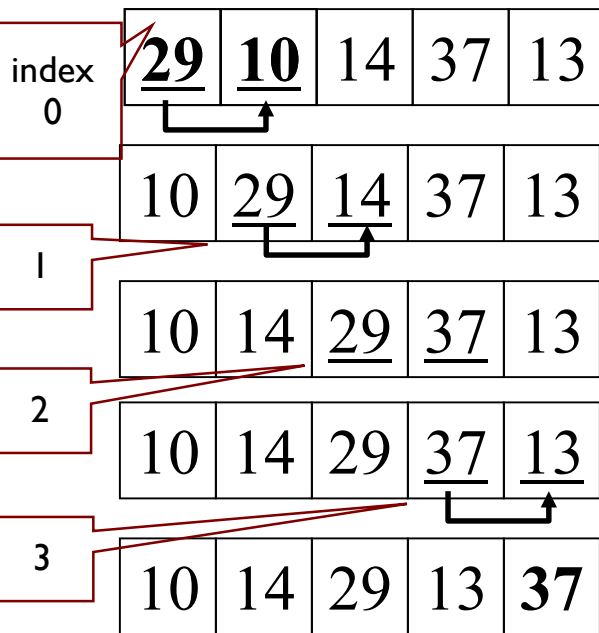
5 Bubble Sort

Bubble Sort

number of
comparisons = 3

First pass

From index: 0 to 3



29 > 10, swap

29 > 14, swap

29 < 37

37 > 13, swap

13 < 37

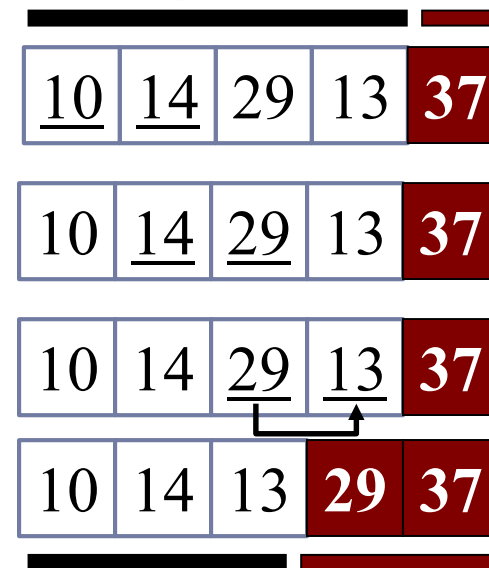
unsorted

sorted

number of
comparisons = 4

Second pass

From index: 0 to 2



10 < 14

14 < 29

29 > 13 swap



5 Bubble Sort

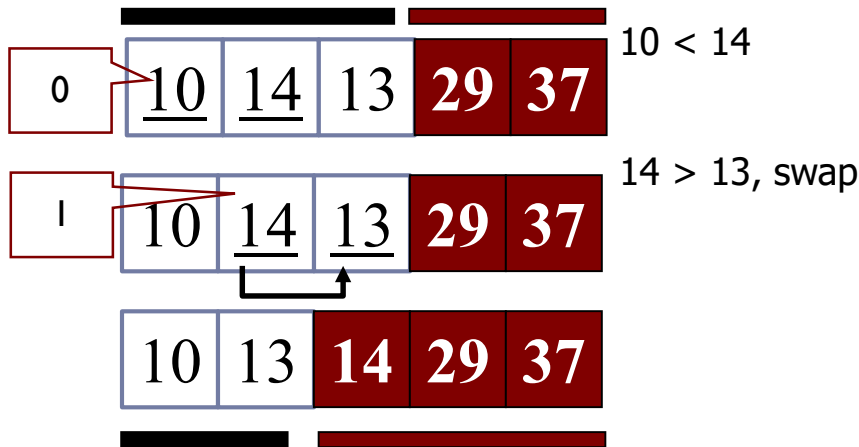
Bubble Sort (con't)

► Continue ...

number of
comparisons = 2

third pass

From index: 0 to 1



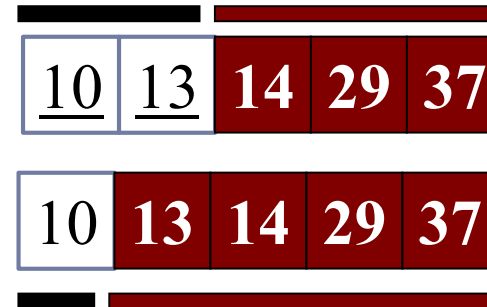
number of
comparisons = 1

4th pass

index: 0

10 < 13

0



► Algorithm

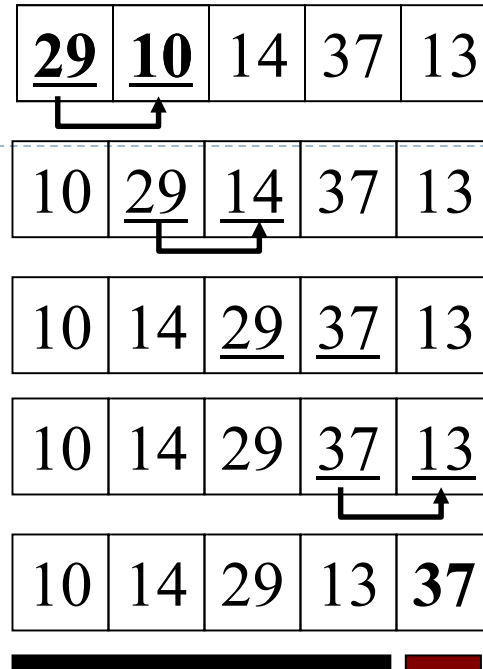
- Iterate through the elements of the list (from 0 to n-1)
 - Compare adjacent items and have the larger element “bubble” to the end of the list by repeated swap operations
 - After the i^{th} iteration, the last i elements of the list will be properly sorted
 - The process is repeated until the unsorted part has a size of 1.



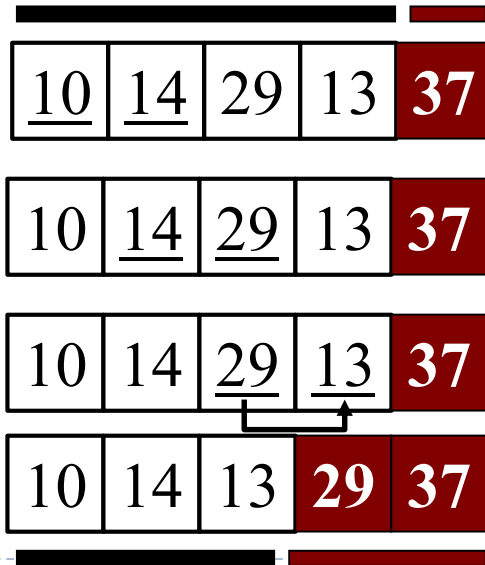
5 Bubble Sort Analysis

pass	Comp.	Swap
1 st	4	3
2 nd	3	1
3 rd	2	1
4 th	1	0

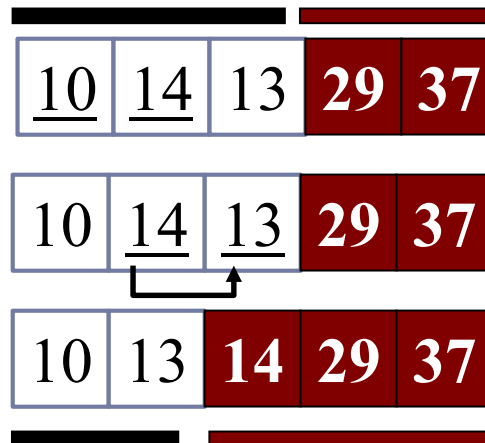
First pass



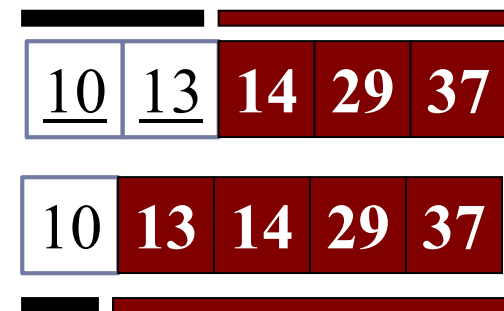
Second pass



third pass



4th pass





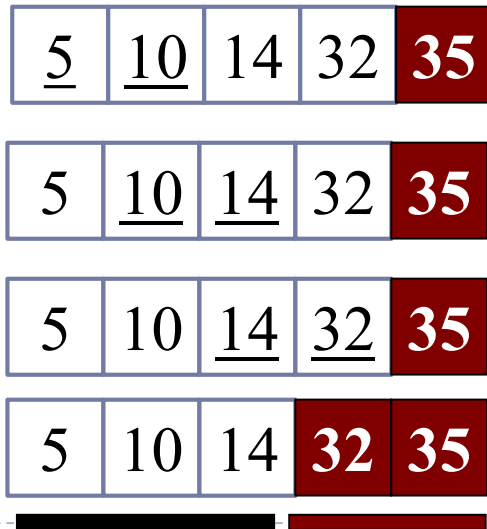
5 Bubble Sort Analysis

► What if the data is already sorted?

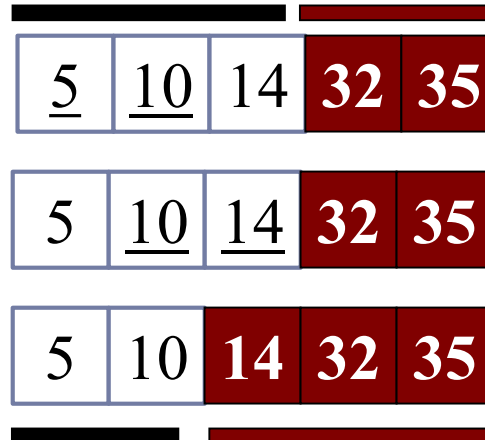
How can we do better?

pass	#comp.	#swaps
1 st	4	0
2 nd	3	0
3 rd	2	0
4 th	1	0

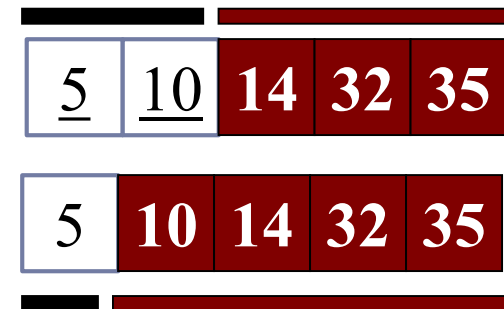
Second pass



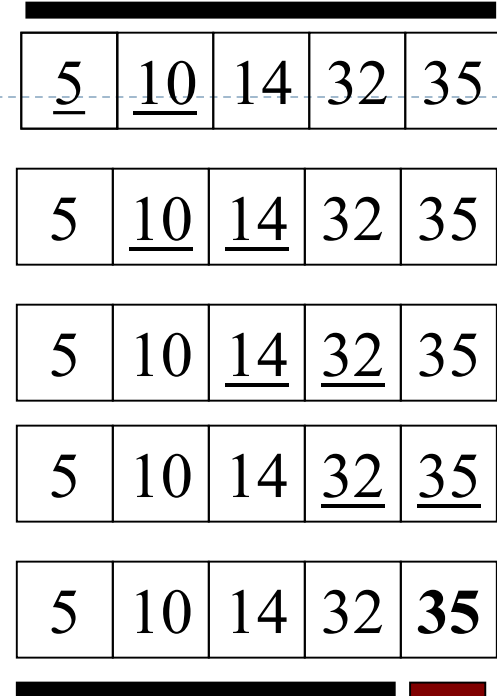
third pass



4th pass



First pass



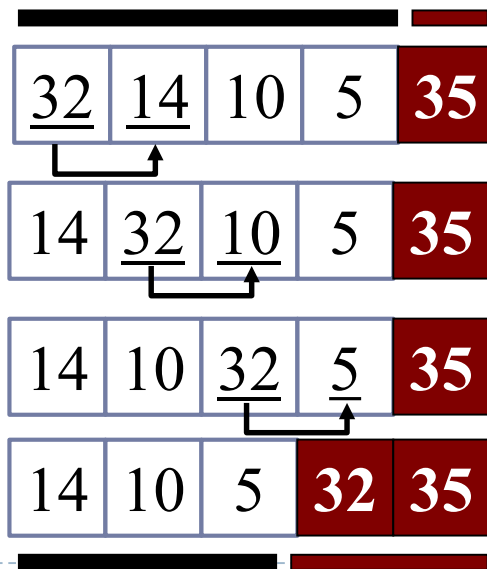


5 Bubble Sort Analysis

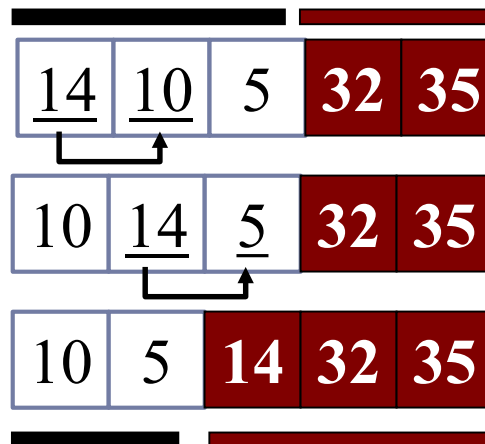
- ▶ What if the data is in reverse order?

pass	#comp.	#swaps
1 st	4	4
2 nd	3	3
3 rd	2	2
4 th	1	1

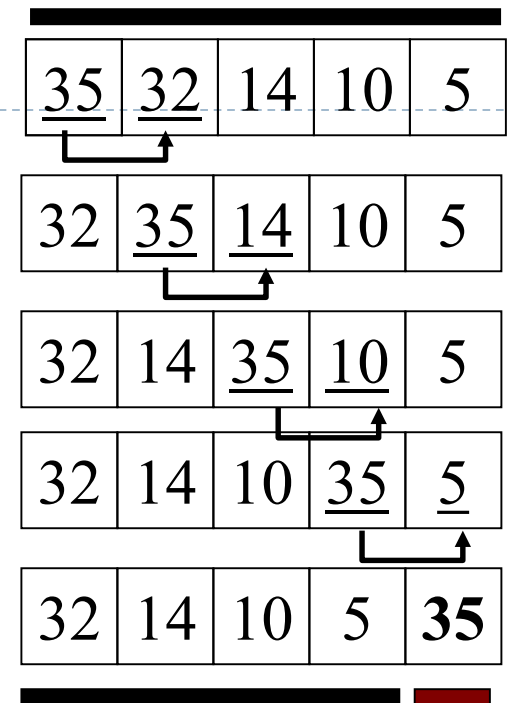
Second pass



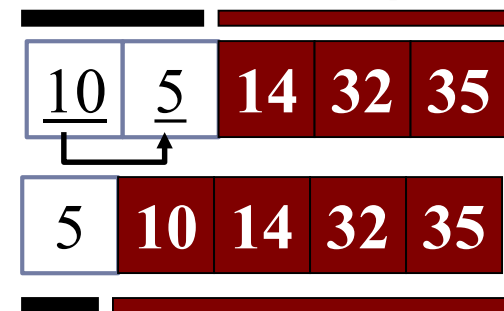
third pass



First pass



4th pass





Animation

- ▶ It works by repeatedly stepping through the list to be sorted, **comparing two items** at a time, and **swapping these two items** if they are in the wrong order.

Animation

6 5 3 1 8 7 2 4

Image source: Wikipedia



5 Bubble Sort Exercise 4



► Sort the following list into ascending order.

► 12, 23, 10, 34, 5

Initial list	12	23	10	34	5
First round of “bubbling”					
Second round of “bubbling”					
Third round of “bubbling”					
Fourth round of “bubbling”					



5 Bubble Sort Analysis

- ▶ What is the worst case Big-O complexity?
 - ▶ $O(n^2)$
- ▶ On each pass, there are **multiple** comparisons and **multiple** swaps
- ▶ Best Case
 - ▶ Bubble Sort uses one pass: $n-1$ comparisons and no swap
 - ▶ Best case complexity is $O(n)$
- ▶ Advantages:
 - ▶ Easy to understand & implement, optimal best case behaviour
- ▶ Disadvantages:
 - ▶ It is generally considered to be the most inefficient sorting algorithm in common practical usage



Exercise 5

- ▶ `swap_adjacent(numbers, index)`
 - ▶ checks two elements:
 - ▶ element at the specified index and the next one
 - ▶ swaps two adjacent elements if they are not in ascending order

```
a_list = [29, 10, 14, 37, 13]  
swap_adjacent(a_list, 3)  
print(a_list)
```

check 37 and 13
=> not in order, swap
[29, 10, 14, 13, 37]

```
a_list = [3, 15, 25, 37, 45]  
swap_adjacent(a_list, 1)  
print(a_list)
```

check 15 and 25
in order -> no change
[3, 15, 25, 37, 45]

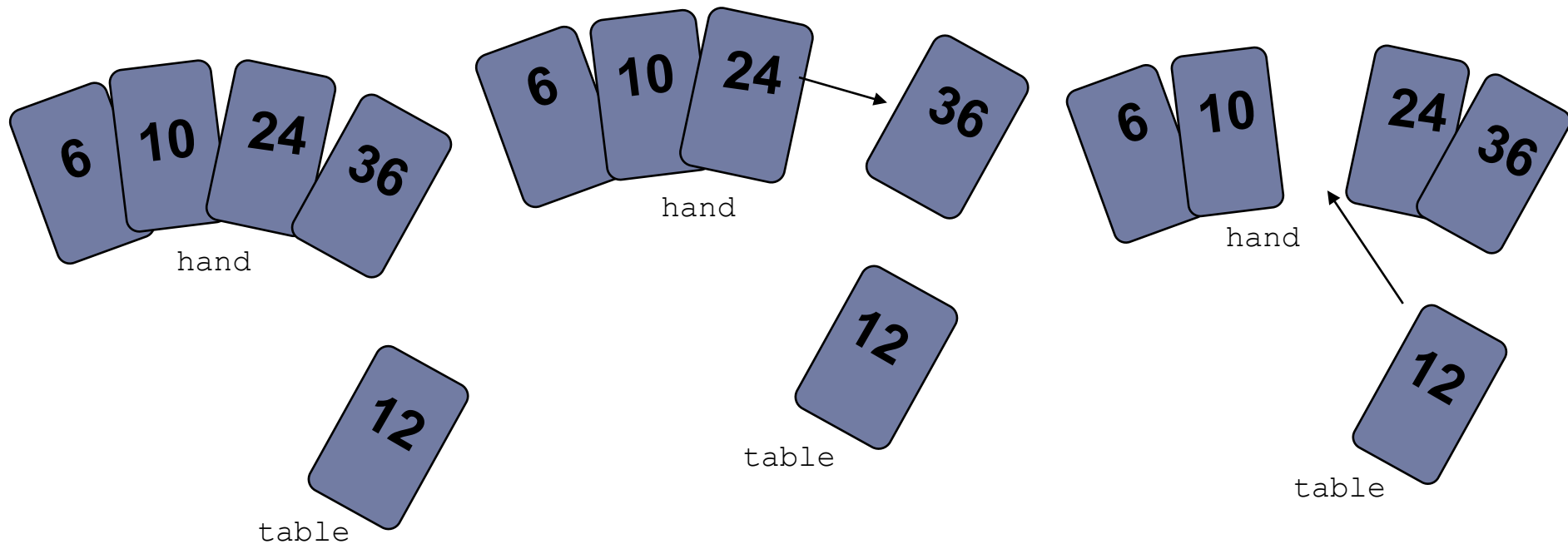


6 Insertion Sort

Insertion Sort

► Illustration

- Take a card from the table and insert it into its proper position in your hand
- To insert the card 12, we need to make room for it by moving first the card 36 and then the card 24





6 Insertion Sort

Insertion Sort

► Basic Idea:

- Insertion Sort partitions the list into two regions: the **sorted** and the **unsorted**.
- At each step, Insertion Sort takes the **first** item of the unsorted region and **places it into its correct position** in the sorted region
- It “grows” the sorted region by inserting subsequent items into the right spot

6 5 3 1 8 7 2 4

Animation

Image source: Wikipedia

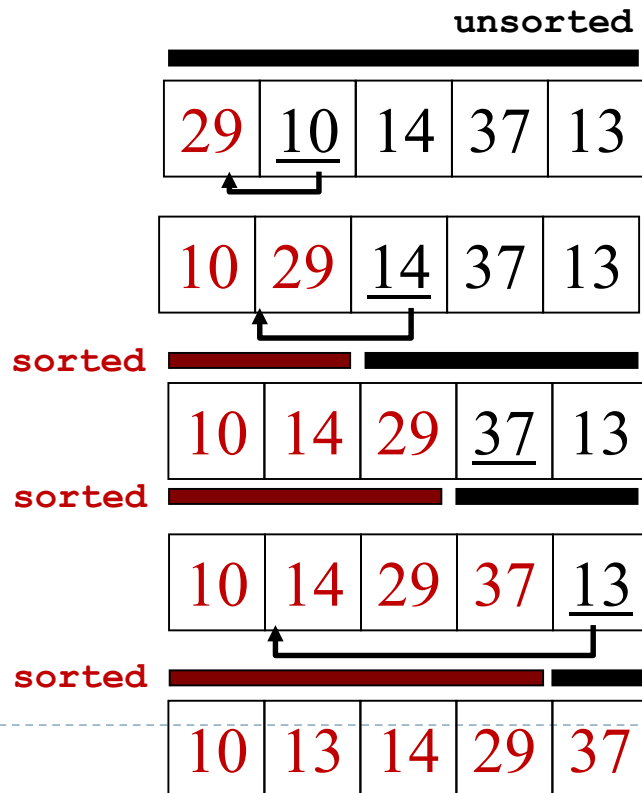


6 Insertion Sort

Insertion Sort

► Algorithm

- Iterate through the elements of the list (from 1 to $n-1$)
 - During pass i , the i^{th} element will be inserted into the proper place
 - This process may require shifting items to make room for the inserted items



Take 10, shift 29 to the right, insert 10

Take 14, shift 29 to the right, insert 14

Take 37, insert 37

Take 13, shift 37, 29, 14 to the right, insert 13



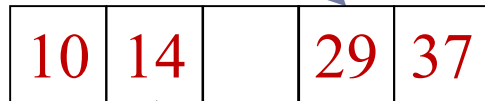
6 Insertion Sort

How to make room

- ▶ For example, to insert 13 into the sorted part of the list we need to store 13 in a temporary variable and to move all the elements which are larger than 13 up one position, then insert 13 into the empty slot.



sorted



temp = 13

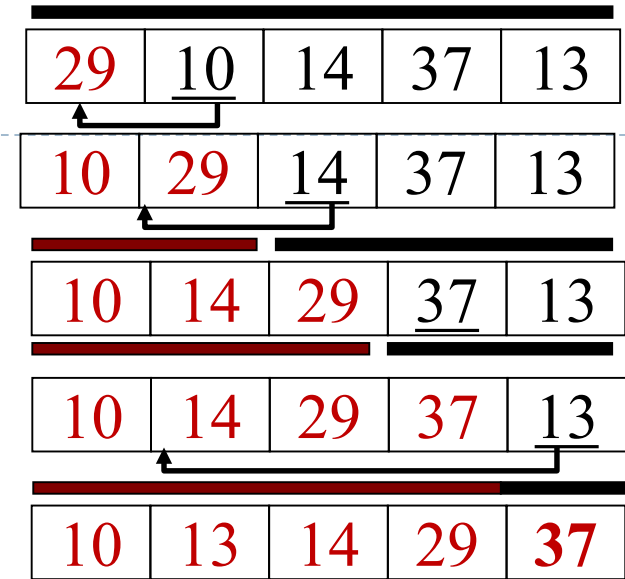




6 Insertion Sort Analysis

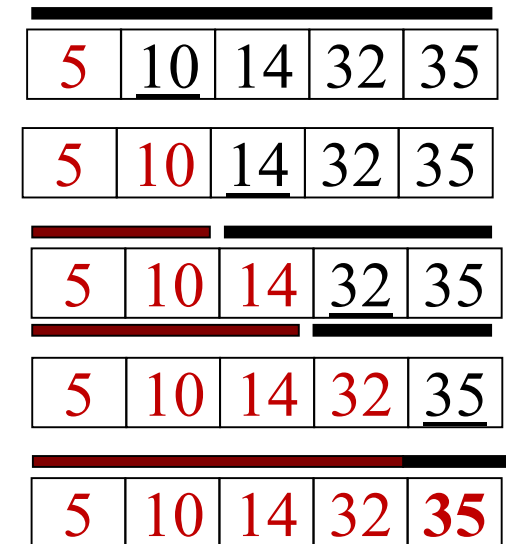
► Example:

pass	#comp.	#shifts
1 st	1	1
2 nd	2	1
3 rd	1	0
4 th	4	3



► What if the data is already sorted?

pass	#comp.	#shifts
1 st	1	0
2 nd	1	0
3 rd	1	0
4 th	1	0





6 Insertion Sort Analysis

- ▶ What is the Big-O complexity?
 - ▶ $O(n^2)$
 - ▶ This performs particularly well if the data is **sorted** or **almost sorted** to begin with, as in that case in each pass only one comparison is performed $\Rightarrow O(n)$
 - ▶ Advantages:
 - ▶ Simple to implement
 - ▶ Efficient on (quite) small data / almost sorted sets
 - ▶ More efficient in practice than most other simple $O(n^2)$ algorithms such as Selection Sort or Bubble Sort
 - ▶ Disadvantage:
 - ▶ Inefficient for large lists with arbitrary data
-



6 Insertion Sort

Exercise 6



► Sort the following numbers into ascending order:

► 20, 10, 15, 25, 6 and 35

Initial list	20	10	15	25	6	35
Take 10 ...	20	<u>10</u>	15	25	6	35



Exercise 7: get_position

- ▶ `get_position(numbers, index)`
 - ▶ copies the element at *index* into a temp variable
 - ▶ searches for the proper position to be inserted
 - ▶ such that all numbers from the beginning to the index position are sorted in ascending order

```
a_list = [10, 14, 29, 12, 56]  
print(get_position(data, 3))
```

index 3 -> 12 -> temp

-> position = 1

[10, 12, 14, 29] are sorted in ascending order

```
a_list = [3, 15, 25, 37, 18]  
print(get_position(data, 4))
```

index 4 -> 18 -> temp

-> position = 2

[3, 15, 18, 25, 37]

Summary “Simple” Sorting Algorithms

	Complexity:	Note:
Selection	Best: $O(n^2)$ Worst: $O(n^2)$	Simple and easy to implement
Bubble	Best: $O(n)$ Worst: $O(n^2)$	Straightforward and simplistic method of sorting data Inefficient in practice
Insertion	Best: $O(n)$ Worst: $O(n^2)$	Efficient for small or mostly-sorted data set

	best case: 5, 10, 14, 32, 35	worst case 35, 32, 14, 10, 5	example 29, 10, 14, 37, 13
selection	10 comparisons, 4 swaps	10 comparisons, 4 swaps	10 comparisons, 4 swaps
bubble	10 c	10 c, 10 s	10 c, 5 s
insertion	4 c	10 c 10 s	8 c, 5 s

- ▶ All sorting algorithms discussed so far had an $O(n^2)$ worst case complexity $\Rightarrow$ In practice for large lists usually too slow
 - ▶ Practical algorithms (e.g. merge sort) have $O(n \log n)$ worst case complexity



7 Binary Search Searching

- ▶ We're often interested in finding something within a set of data
 - ▶ What is the index of the letter 'a' in the sentence "The quick brown fox jumps over the lazy dog?"
 - ▶ Does this website contain the word "map"?
 - ▶ What is the definition of the word "pseudocode"?
 - ▶ Remember Selection Sort:
 - ▶ What is the largest item in a list?
 - ▶ What is the best move I should make in a game of chess?
- ▶ All of these are possible search problems



Linear Search

- ▶ Simplest search function
- ▶ Look at items in the data set one by one, and see if they match
- ▶ $O(n)$ worst case runtime complexity
- ▶ For example find the (first) location of the value C

A	B	C	D	E	F	G	Check index 0
A	B	C	D	E	F	G	Check index 1
A	B	C	D	E	F	G	Check index 2

- ▶ Note: we may also want to find ALL locations of C, or C may not be contained in the list ...



Binary Search

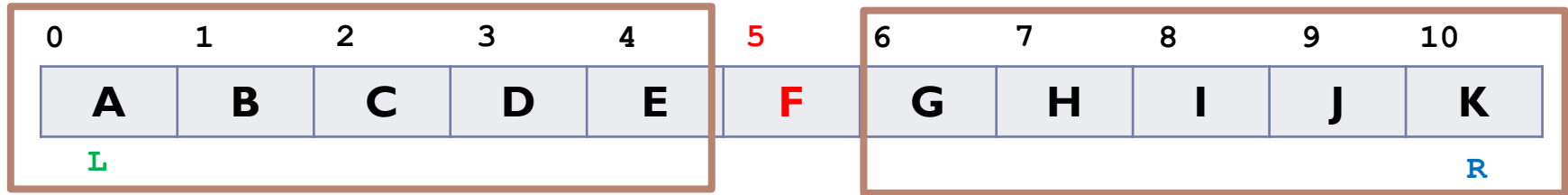
- ▶ What if the data set comes pre-sorted? (Or we can sort it?)
 - ▶ Look at the **middle item** in the data set
 - ▶ Is it the one we're after?
 - ▶ If yes: great! Return it, we're finished
 - ▶ If no:
 - ▶ If it's smaller:
 - ▶ The NEW data set to look at is now the values from the *start to the middle*
 - ▶ If it's larger:
 - ▶ The NEW data set to look at is now the values from the *middle to the end*
 - ▶ Repeat looking at the middle between start and end of respective NEW data set
-



7 Binary Search

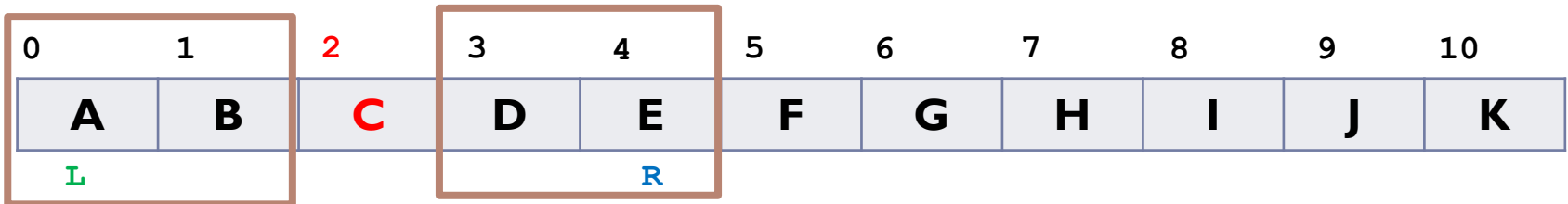
Binary Search – Example 1

- Find the location of the value 'D'



Left = 0 Right = 10 $\Rightarrow$ middle index 5

$D < F$



Left = 0 Right = $5 - 1 = 4$ $\Rightarrow$ middle index 2

$D > C$



7 Binary Search

Binary Search

- Find the location of the value 'D'

0	1	2	3	4	5	6	7	8	9	10
A	B	C	D	E	F	G	H	I	J	K
			L	R						

Left = 2 + 1 Right = 4 middle index = 3, found D!

- Note that with every check we cut the problem size in half.
 - Our problem size $n=11$ resulted in an algorithm requiring 3 steps to locate "D"
 - In general, for problem size n , runtime is proportional to $\log_2$ of n !
- We have a worst case runtime upper bounded by $O(\log n)$



7 Binary Search

Binary Search – Example 2

- Find the location of the value '5'

0	1	2	3	4	5	6	7
1	3	4	6	7	8	9	10
L				R			

Left = 0 Right = 7 $\Rightarrow$ middle index 3

$5 < 6$

0	1	2	3	4	5	6	7
1	3	4					
L			R				

Left = 0 Right = 3 - 1 = 2 $\Rightarrow$ middle index 1

$5 > 3$



7 Binary Search

Binary Search – Example 2

- Find the location of the value '5'

0	1	2
1	3	4
		L R

Left = 2 Right = 2, => middle index 2
5 > 4



Left = 3 Right = 3 - 1 = 2 => left > right => STOP



7 Binary Search

Binary Search

- ▶ Because we split the search space in half at each step, doubling the size of the input only adds one extra step.
- ▶ Therefore, Binary Search is $O(\log n)$
- ▶ If the data is not sorted, then the overall cost of searching will be $O(\text{sorting algorithm} + \log n)$
 - ▶ E.g. if we use Selection Sort we get $O(n^2 + \log n)$
 - ▶ Because Big-O is dominated by the less efficient part, this is $O(n^2)$
- ▶ If we have to search for multiple values, we can re-use the sorted version of the list, so the more times we search, the less relative overhead from sorting!
 - ▶ Overhead of sorting gets amortized with repeated searches!

Binary Search Exercise 8

- A) numbers: [5, 10, 15, 20, 23, 32, 46, 89], search for “10”

0	L	1	2	3	4	5	6	7	R
5	10	15	20	23	32	46	89		

Left = 0 Right = 7 We check index 3, value = 20, =>left

0	1	2	3	4	5	6	7
5	10	15	20	23	32	46	89

- B) search for 32

Left = 0 Right = 7 We check index 3, value = 20, =>right

0	L	1	2	3	4	5	6	7	R
5	10	15	20	23	32	46	89		

0	1	2	3	4	5	6	7
5	10	15	20	23	32	46	89



Binary Search Exercise 8

- C) numbers: [5, 10, 15, 20, 23, 32, 46, 89], search for “12”

0	L	1	2	3	4	5	6	7	R
5	10	15	20	23	32	46	89		

Left = 0 Right = 7 We check index 3, value = 20, =>left

0	L	1	2	R	3	4	5	6	7
5	10	15	20	23	32	46	89		

0	1	2	3	4	5	6	7
5	10	15	20	23	32	46	89



Exercise 9

- ▶ `print_direction(numbers, target)`
 - ▶ calculates the `mid_index`
 - ▶ prints “LEFT” if target is less than the value at `mid_index`
 - ▶ prints “RIGHT” if target is greater than the value at `mid_index`
 - ▶ prints “FOUND” if target is the same as the value at `mid_index`
 - ▶ Note: you can assume that *target* must exist in the parameter list.

```
a_list = [10, 12, 14, 29, 39]
print_direction(a_list, 12)
a_list = [3, 15, 25, 37, 48, 57]
print_direction(a_list, 37)
a_list = [3, 15, 25, 37, 48, 57]
print_direction(a_list, 25)
```

```
mid_index = 2
LEFT
```

```
mid_inex = 2
RIGHT
```

```
mid_inex = 2
FOUND
```